

# Template - Modelo ADDIE (Etapa de Análise)

---

**Propósito:** Criação de Curso Inovador em Computação

## Compreensão do Problema Educacional:

- Tema do curso escolhido: Debugging para Programadores Iniciantes com apoio de Inteligência Artificial Generativa e Gamificação
- Contexto (acadêmico, corporativo, híbrido, etc.): Acadêmico, com possibilidade de aplicação híbrida em cursos introdutórios de programação (CS1), bootcamps e programas de formação inicial em computação.

👉 **Problema educacional que o curso pretende resolver:** Programadores iniciantes frequentemente apresentam dificuldades em compreender mensagens de erro, localizar falhas no código e desenvolver estratégias sistemáticas de depuração. Em muitos casos, o ensino de programação prioriza a escrita do código correto, mas dedica pouca atenção ao processo de debugging, gerando frustração, baixa autoconfiança e desengajamento. De forma complementar, o uso crescente de ferramentas de IA generativa, como ChatGPT, tem incentivado práticas passivas, nas quais os estudantes apenas copiam soluções prontas sem compreender o raciocínio envolvido. O curso busca transformar o debugging em um processo investigativo, ativo e motivador, utilizando gamificação e IA generativa com foco em apoio pedagógico, saindo da prática de apenas realizar a entrega automática de respostas.

---

## 1. Caracterização do Público-Alvo

**Objetivo:** Compreender profundamente o aprendiz.

- Perfil (formação, nível, área): Estudantes iniciantes em programação, principalmente alunos de cursos introdutórios de computação, engenharia, sistemas de informação ou áreas correlatas.
- Experiência prévia em computação: Conhecimento básico de lógica de programação e primeiros contatos com linguagens como Python, C ou Java.
- Motivações para aprender: Desejo de aprender programação, melhorar desempenho acadêmico, desenvolver autonomia para resolver problemas e aumentar confiança ao programar.
- Dificuldades comuns: Interpretação de mensagens de erro, identificação da origem dos bugs, ansiedade diante de falhas no código, dependência excessiva de colegas ou ferramentas de IA e dificuldade em desenvolver raciocínio lógico estruturado.
- Contexto (tempo disponível, acesso a recursos, etc.): Aprendizes com tempo limitado para estudo extraclasse, geralmente utilizando notebooks pessoais e recursos online gratuitos. Muitos conciliam estudos com outras disciplinas ou atividades profissionais.

👉 **Persona construída pela equipe:** Lucas, 19 anos, estudante do primeiro período de Ciência da Computação, consegue escrever pequenos programas, mas sente grande dificuldade quando surgem erros que ele não entende. Costuma buscar respostas prontas em fóruns ou no ChatGPT e frequentemente se sente frustrado ao não conseguir identificar o motivo de uma falha no código. Apesar disso, demonstra interesse em aprender de forma prática e interativa.

---

## 2. Análise de Necessidades (Learning Gap)

**Objetivo:** Identificar a lacuna entre o estado atual e o desejado.

- O que os aprendizes já sabem (conhecimentos prévios): Conceitos básicos de programação, estruturas condicionais, repetição, variáveis e funções simples.
- O que eles precisam saber/fazer ao final do curso: Conhecer estratégias sistemáticas de debugging, compreender melhor mensagens de erro, aprender a testar hipóteses sobre possíveis falhas no código e utilizar a IA generativa como apoio pedagógico, sem depender exclusivamente de respostas prontas.
- Principais lacunas identificadas: Dificuldade em interpretar erros, pouca familiaridade com estratégias de depuração, tendência a buscar soluções prontas e insegurança para analisar problemas de forma autônoma.
- Evidências do gap (dados, relatos, experiências): Relatos frequentes de frustração em disciplinas introdutórias, evasão em cursos de programação, dificuldades observadas em atividades práticas e estudos recentes sobre dependência excessiva de IA generativa em programação.

👉 **Síntese do Gap:** Os estudantes já conseguem produzir códigos básicos, mas ainda têm pouca familiaridade com estratégias para compreender, investigar e corrigir erros. O curso busca apoiar esse processo, apresentando formas mais estruturadas, guiadas e investigativas de lidar com o debugging.

---

## 3. Análise de Tarefas

**Objetivo:** Mapear o que o aprendiz precisa ser capaz de fazer.

- Tarefas-chave que o aprendiz deverá executar:
  - Tarefa 1: Interpretar mensagens de erro e identificar possíveis causas do problema.
  - Tarefa 2: Aplicar estratégias sistemáticas de debugging utilizando testes, inspeção de código e ferramentas de apoio.
  - Tarefa 3: Utilizar IA generativa de forma crítica e guiada para apoiar o processo de depuração sem depender exclusivamente da ferramenta.
- Conhecimentos necessários:
  - Estruturas básicas de programação
  - Conceitos de lógica computacional

- Tipos comuns de erros (sintaxe, lógica e execução)
- Habilidades necessárias:
  - Pensamento lógico
  - Resolução de problemas
  - Leitura e interpretação de código
  - Teste e validação de hipóteses
- Atitudes/comportamentos esperados:
  - Persistência diante de erros
  - Curiosidade investigativa
  - Uso responsável da IA
  - Participação ativa nas atividades

#### 👉 Complexidade das tarefas:

☐ Baixa ☒ Média ☐ Alta

Justifique: Embora os conceitos iniciais sejam básicos, o processo de debugging exige raciocínio investigativo, interpretação contextual e tomada de decisão contínua, o que demanda prática e acompanhamento pedagógico.

---

## 4. Análise Instrucional

**Objetivo:** Definir o tipo e nível de instrução necessários.

- Tipo de aprendizagem predominante:  
☐ Conceitual ☐ Procedimental ☐ Atitudinal ☒ Mista
- Nível de profundidade necessário:  
☒ Introdutório ☐ Intermediário ☐ Avançado
- Necessidade de prática:  
☐ Baixa ☐ Moderada ☒ Alta
- Estratégias instrucionais mais adequadas (hipóteses):  
☐ PBL ☐ Projetos ☒ Estudos de caso ☒ Simulação ☐ Aula expositiva ☒ Outra:  
Gincana gamificada em equipes

#### 👉 Justificativa da abordagem instrucional:

As estratégias foram selecionadas para transformar a relação do iniciante com o erro, deslocando o foco da frustração para a investigação guiada. A aprendizagem mista é adequada porque combina compreensão conceitual, prática procedural e mudança de postura diante do debugging. Os estudos de caso permitem analisar códigos com erros reais ou simulados, enquanto as simulações colocam o estudante em cenários práticos de depuração, com apoio da IA como ferramenta de consulta crítica, e não como fonte de respostas prontas. A dinâmica de gincana gamificada em equipes pode tornar o processo mais engajador, promovendo colaboração, troca de estratégias e resolução progressiva de desafios, em uma lógica semelhante a eventos práticos de inovação, como hackathons, mas adaptada ao nível introdutório dos participantes.

---

## 5. Levantamento de Competências

**Objetivo:** Definir o que será desenvolvido no curso.

- Competências técnicas (**hard skills**):
  - Interpretação de mensagens de erro
  - Aplicação de técnicas de debugging
  - Leitura e análise de código
  - Uso pedagógico de IA generativa
  - Resolução estruturada de problemas computacionais
- Competências socioemocionais (**soft skills**):
  - Persistência
  - Autonomia
  - Pensamento crítico
  - Autorregulação da aprendizagem
  - Confiança para resolver problemas

### Padrões de desempenho esperados:

O estudante deverá ser capaz de identificar erros de forma estruturada, justificar suas hipóteses de correção, utilizar IA como apoio investigativo e resolver problemas simples e intermediários de programação sem depender exclusivamente de soluções prontas.



### Reflexão Final

 Principal insight sobre o aprendiz: O maior desafio do aprendiz iniciante não está apenas em escrever código, mas em compreender e investigar erros sem se sentir incapaz ou dependente de respostas externas.

 Principais implicações para o design do curso: O curso precisa priorizar prática guiada, feedback imediato, redução da ansiedade diante dos erros e atividades que incentivem investigação ativa em vez de apenas fornecer respostas prontas.

 Riscos caso a análise seja ignorada: O curso pode se tornar excessivamente técnico, desmotivador e pouco aderente às dificuldades reais dos iniciantes, aumentando a dependência de IA e reduzindo a autonomia do estudante.

Marque se sua análise:

- (X) Está centrada no aprendiz (não no conteúdo)
- (X) Identifica claramente o gap de aprendizagem
- (X) Considera contexto real de aplicação
- (X) Integra competências técnicas e socioemocionais
- (X) Evita soluções genéricas
- (X) Gera insumos claros para o design do curso

Refletindo sobre essa atividade:

- O que aprendemos sobre o aprendiz que não sabíamos antes?
  - Percebemos que muitos iniciantes conseguem escrever pequenos trechos de código, mas não possuem estratégias claras para lidar com erros e frequentemente associam debugging a fracasso pessoal.
- O que foi mais difícil nesta análise?
  - Equilibrar o foco entre aspectos técnicos da programação e fatores emocionais, como ansiedade, frustração e dependência de IA.
- Como o trabalho em equipe contribuiu para a compreensão do problema?
  - A troca de perspectivas entre integrantes com diferentes formações ajudou a ampliar a visão sobre o problema, considerando tanto aspectos pedagógicos quanto técnicos e motivacionais.
- Como esta análise contribui para tornar o curso mais relevante, significativo e inovador?
  - A análise permite que o curso seja desenhado a partir das dificuldades reais dos aprendizes, incorporando IA e gamificação não apenas como tecnologia, mas como estratégias pedagógicas voltadas ao desenvolvimento da autonomia e do pensamento crítico.